

SIMATS ENGINEERING

Department of Computer Science and Engineering

Course Code:	CSA1512	Assessment:	CO5 AT2
Course Name:	Cloud Computing and Big Data Analytics	Type:	Operations Optimization
CO Statement:	Apply big data storage, integration, Hadoop, HDFS, and MapReduce concepts for distributed data processing		

Part A: Production Hadoop Operations – Lesson Notes

Monitoring & Observability

A production Hadoop cluster generates thousands of metrics. Monitoring means: (1) collecting these metrics, (2) storing them, (3) displaying dashboards, (4) setting thresholds to trigger alerts.

Component	Key Metrics	Normal Range	Alert Threshold
NameNode	Heap usage, Live/Dead DataNodes, Block count, Edit log size	< 70% heap	> 85% heap
DataNode	Disk usage, Block count, Network I/O, JVM heap	< 80% disk	> 92% disk
MapReduce	Job completion rate, Task failure %, Shuffle sort time	< 5% failure	> 20% failure
HDFS	Under-replicated blocks, Missing blocks, Block replica imbalance	0 under-replicated	> 10 blocks

Lesson Note: Why Monitoring Matters

Without monitoring, you only discover problems when users complain. NameNode heap at 95% → crash → cluster down → data unavailable. Monitoring alerts at 85% gives you time to act (add DataNodes, scale up, optimize)

Backup & Disaster Recovery

NameNode stores the file system namespace (file names, hierarchy, replica locations) in memory and on disk (fsimage + edit logs). If NameNode crashes and no backup exists, you lose the entire namespace — irrecoverable.

Key Concepts:

1. RTO (Recovery Time Objective): How long can the cluster be down? 1 hour? 1 minute?
2. RPO (Recovery Point Objective): How much data loss is acceptable? 1 hour? 1 day?

Backup Strategy	RTO	RPO	Cost	Best For
No backup (risky)	∞ (unrecoverable)	∞	\$0	Dev/test only

Daily full backup to external disk	1–2 hours	24 hours	\$100/month	Dev, non-critical
Hourly incremental + offsite replication	15 minutes	1 hour	\$500/month	Production
HA (active/standby NameNode + ZK quorum)	< 1 minute	0 (near-instant)	\$1000+/month	Critical production

High Availability (HA) Design

HA eliminates the single NameNode as a single point of failure. Setup: Active NameNode + Standby NameNode + ZooKeeper quorum (3 nodes). When Active fails, Standby takes over automatically in < 1 minute. Requires shared storage (NFS/QJM) for edit logs.

Cost Optimization

Monthly cluster cost = Hardware + Power + Cooling + Ops

Cost Component	Typical \$	Control Method
Hardware (50 nodes × \$500)	\$25,000 one-time	Buy less, or decommission slow nodes
Annual power & cooling (50 nodes × \$1000)	\$50,000/year	Efficient hardware, underutilization
Operations & support	\$10,000/month	Automation, monitoring, runbooks
Compression (Gzip, Snappy)	\$–5,000/month savings	Reduced storage, higher CPU
Tiering (move old logs to cold storage)	\$–3,000/month savings	Archive logs > 30 days to S3

Scaling Strategies

Horizontal scaling: Add more DataNodes (linear cost, complexity)

Vertical scaling: Upgrade existing nodes (cap at 10–20TB/node practical limit)

NameNode Capacity Constraint

NameNode heap ∝ number of files. Rule of thumb: 1GB heap ≈ 1M files. If 100M files needed, you need 100GB NameNode heap — impractical. Solutions: (1) Tiering (archive old data), (2) Federation (multiple NameNodes), (3) Consolidate small files.

Part B: Main Scenario – Optimize a 50-Node Production Cluster

Current State Assessment

Medi Queue AI's backend runs a 50-node HDFS cluster processing 10TB/day of queue, appointment and no-show-prediction event data across all onboarded hospital branches. Current metrics:

Metric	Current Value	Status
--------	---------------	--------

NameNode heap usage	85%	RISKY – near crash threshold
DataNode disk utilization	92%	RISKY – no space for replication
Monthly cost	\$50,000	High, no optimization
Last backup	2 weeks ago	UNACCEPTABLE RPO
DataNode failure recovery time	8 hours	UNACCEPTABLE RTO
Monitoring setup	Manual spreadsheets	No real-time alerts
HA setup	None (single NameNode)	Single point of failure

Problem Summary

The cluster is on the edge of failure. NameNode heap crash is imminent. No automated failover. Slow recovery from node failures. Costs unchecked. Backup is stale.

Step 1: Design Monitoring Dashboard

Create a monitoring strategy: what to track, alert thresholds, and dashboard design.

1a. NameNode Metrics

Metrics to track:

- Heap usage (%)
- Live/Dead DataNodes (count)
- Block count (total)
- Edit log size (MB)

Metric	Green	Yellow	Red
NameNode heap %	< 60%	60–80%	> 80%
Dead DataNodes	0	1–2	> 2
Under-replicated blocks	0	< 100	> 100

Fill in your thresholds and actions:

NameNode heap — Green (<60%): no action, routine monitoring. Yellow (60–80%): start a capacity-planning review (file-growth trend, consider tiering/consolidation); daily check, no page. Red (>80%): page the on-call engineer immediately, pause non-essential file creation if possible, begin emergency tiering/consolidation, and accelerate the Federation timeline. Dead DataNodes — Green (0): normal. Yellow (1–2): investigate node health, confirm re-replication is proceeding, no page. Red (>2): page on-call, check for a correlated rack/network failure, verify RF is being restored across the remaining racks. Under-replicated blocks — Green (0): normal. Yellow (<100): monitor, usually self-heals within minutes. Red (>100): page on-call — signals a large-scale failure or a stuck replication pipeline. Dashboard: one Grafana-style board (Prometheus + JMX exporters on NameNode/DataNodes) showing these as traffic-light tiles plus a 24-hour heap%/block-count trend, since Medi Queue AI's queue data is patient-care-critical and problems should be caught before hospital staff notice the app failing.

Step 2: Plan Backup & Disaster Recovery

Current state: Last backup 2 weeks ago → RPO of 14 days is unacceptable.

2a. Backup Strategy Design a backup plan:

Decision	Option A	Option B	Option C
Backup frequency	Daily	Hourly	Real-time (HA)
Storage location	On-cluster NFS	Off-site S3	Shared QJM
RTO	2–4 hours	1 hour	< 1 minute
RPO	24 hours	1 hour	Near-zero
Cost/month	\$200	\$1000	\$2000+ HA infra

Your recommendation: Choose the option that balances cost and business needs.

Recommended now: Option C-equivalent — hourly incremental backup + off-site replication (RTO 1 hour, RPO 1 hour, ~\$500/month) — to immediately replace the current 2-week-stale backup, with full HA (RTO<1 min, RPO≈0, \$1000+/month) planned as the Step 5 target end-state. Justification: Medi Queue AI's data directly supports same-day patient scheduling, so even a 24-hour RPO (Option B) is too risky — losing a day of queue/no-show history could force re-confirming appointments hospital-wide — while jumping straight to full HA's cost is a bigger investment than is needed to fix today's 14-day RPO emergency. Hourly incremental + off-site replication closes the dangerous gap now at moderate cost.

Step 3: Cost Optimization Analysis

Current: 50 nodes × \$1000/year ops = \$50K/year. Target: Reduce by 20% to \$40K/year.

Optimization	Annual Savings	Impact	Your Choice?
Compression (Gzip logs)	\$5,000	Reduced storage, higher CPU	Yes / No
Tiering (archive > 30 days to cold)	\$3,000	Slower access to old logs	Yes / No
Decommission 5 slow nodes	\$5,000	Less capacity, need to rebalance	Yes / No
Auto-scaling (scale down off-peak)	\$2,000	Reduced peak hours ops	Yes / No

Calculate total savings from your selections and new monthly cost:

Selected: Compression (Gzip logs) — Yes, \$5,000/year. Tiering (archive >30 days to cold) — Yes, \$3,000/year. Auto-scaling (scale down off-peak) — Yes, \$2,000/year. Decommission 5 slow nodes — No (skipped: Medi Queue AI is patient-care-critical, and cutting capacity risks under-provisioning during a regional demand spike, e.g. flu season). Total selected savings = \$5,000 + \$3,000 + \$2,000 = \$10,000/year, exactly meeting the 20% target (\$50,000 → \$40,000/year). New monthly cost = \$40,000 ÷ 12 ≈ \$3,333/month (down from ~\$4,167/month).

Step 4: Address NameNode Heap Pressure

Current heap: 85% at 50M files. At current growth (1M files/month), heap will exceed 90% in ~2 months → crash risk.

Option	Action	NameNode Heap Impact	Timeline
Tiering	Archive logs > 30 days (removes old files)	Reduces heap by ~10%	1 month
Consolidate	Merge small files into larger blocks	Reduces heap by ~15%	Ongoing
Scale	Add second NameNode (Federation)	Splits namespace across 2 NameNodes	2 months setup

Which solution do you recommend? Why?

Recommended: start Consolidate immediately (merge small per-event queue logs into larger rolling files) — fastest to deploy and the largest single reduction (~15% heap), attacking the root cause of file-count growth. Run Tiering in parallel (archive logs >30 days, ~10% heap reduction, ~1 month), since Medi Queue AI's oldest queue logs are rarely queried after a month. Together these buy roughly 2–3 extra months of headroom. At the same time, start the 2-month Federation (second NameNode) project now rather than waiting for the crisis — consolidation and tiering only slow file growth, they don't remove the single-NameNode ceiling, and Medi Queue AI's branch count keeps increasing.

Step 5: Implement High Availability (HA)

Current: Single NameNode (SPOF). Proposed: HA with Standby.

Component	Quantity	Purpose
Active NameNode	1	Primary metadata server
Standby NameNode	1	Automatic failover replica
ZooKeeper quorum	3 or 5	Coordinates HA election
Shared edit log storage	NFS or QJM	Both NNs access same metadata

Failover process: Active NameNode fails → ZK detects (30 sec) → Standby promotes (30 sec) → Clients reconnect. Total RTO < 1 minute.

Estimate HA setup cost and timeline:

One-time setup: a dedicated Standby NameNode server plus reusing/adding machines for the 3–5-node ZooKeeper quorum ≈ \$500–\$1,000 hardware, plus ~1–2 weeks of a Hadoop admin's time to configure QJM shared edit logs, ZKFC, and run failover drills. Ongoing cost: roughly \$1,000+/month (Table 2's HA row) for the standby hardware, ZK quorum nodes, and the added operational vigilance HA requires. Timeline: 2–3 weeks initial setup/configuration + 1–2 weeks of deliberate failover-drill testing before trusting it in production ≈ 4–5 weeks total — comfortably ahead of the ~2-month heap-crash deadline from Step 4 if started now.

Part C: Mini-Problems

Mini-Problem 1: DataNode Failure Scenario

A DataNode holding 12TB of data fails (hardware crash). HDFS detects the failure and begins re-replicating blocks from surviving replicas. Current network capacity: 1 Gbps per DataNode.

Calculate re-replication time and estimate when cluster is back to normal replication factor.

1 Gbps $\approx$ 125 MB/s. 12 TB $\approx$ 12,582,912 MB. Single-link time $\approx$ $12,582,912 \div 125 \approx 100,663$ seconds $\approx$ 28 hours. In practice HDFS re-replicates from/to several peer DataNodes in parallel, so real recovery is likely closer to 5–6 hours if spread across 5–6 peers at once. The cluster is back to its normal replication factor once every block from the failed node's 12TB has a fresh replica elsewhere — worst case $\sim$ 28 hours, realistic case single-digit hours — during which Medi Queue AI's queue data runs at reduced redundancy, reinforcing why HA and fast inter-rack bandwidth both matter.

Mini-Problem 2: NameNode Heap Projection

Current state: 50M files, 60% heap usage. Growth rate: 1.5M files/month. If file count grows linearly, when will NameNode heap exceed 90%? When should you add a second NameNode (Federation)?

50M files = 60% heap, so 100% heap capacity $\approx$ $50M \div 0.60 \approx 83.3M$ files. 90% heap $\approx$ $0.90 \times 83.3M \approx 75M$ files — 25M files above today. At 1.5M files/month, that's $25M \div 1.5M \approx 16.7$ months away. Recommendation: don't wait for 90% — trigger Federation planning once heap reaches a safer 75–80% ($\approx$ 62.5–66.7M files), which at this growth rate is only 8.3–11.1 months out, so Federation setup should begin well before the 1-year mark to be live before the 90% danger zone.

Mini-Problem 3: Compression Trade-off

Implement Gzip compression on 20TB of historical logs. Storage reduction: 3:1 (20TB $\rightarrow$ 6.67TB). But: CPU overhead for decompression during MapReduce jobs increases job time by 15%.

Is compression worth it economically? Calculate storage cost savings vs. increased compute cost.

Storage: 20TB $\rightarrow$ 6.67TB (3:1). At \$50/TB/month (per Step 3): before = $20 \times \$50 = \$1,000$ /month; after $\approx$ $6.67 \times \$50 \approx \333 /month; savings $\approx$ $\$667$ /month $\approx$ $\$8,000$ /year. Compute: a 15% increase in MapReduce job time on the affected jobs adds roughly tens of dollars a month in extra compute/ops time if those jobs currently cost a few hundred dollars a month — an order of magnitude smaller than the storage saving. Decision: yes, compress — the 3:1 storage saving clearly dominates the 15% compute penalty, provided a splittable codec (Snappy/bzip2, not plain Gzip) is used so MapReduce parallelism isn't also lost.

Mini-Problem 4: Tiering Strategy

Move logs older than 30 days to cold storage (AWS S3 Glacier, \$1/TB/month). Current hot storage cost: \$50/TB/month. Hot tier capacity: 20TB. Cold tier capacity: unlimited.

Calculate break-even point: after how many days of cold storage does it become cheaper than hot?

Hot: \$50/TB/month $\approx$ $\$1.67$ /TB/day. Cold: \$1/TB/month $\approx$ $\$0.033$ /TB/day. Cold is already $\sim$ 50 $\times$ cheaper per TB per month, so on pure storage cost the crossover is immediate — day 0, not some later break-even day. The real reason Medi Queue AI waits 30 days before tiering isn't storage-cost break-even, it's access-pattern economics: cold tiers like S3 Glacier charge retrieval fees and take minutes-to-hours to restore a file, so data staff might still need quickly (recent queue history, this week's no-show follow-ups) stays on the fast/expensive hot tier, while data nobody queries routinely after $\sim$ 30 days moves to the tier that's cheaper since its slow retrieval no longer matters.

Part D: Problem Bank – 4 Additional Scenarios

Scenario A: Multi-Datcenter Failover

Company wants to replicate the 50-node cluster to a second datacenter 100km away for disaster recovery. Design cross-datacenter replication strategy, network bandwidth requirements, and cost implications. Should data be synchronously or asynchronously replicated? What is RTO/RPO?

Replicate Medi Queue AI's HDFS data asynchronously to the second datacenter 100km away — synchronous replication over that distance adds real write latency (network round-trip on the order of milliseconds-to-tens-of-ms) that would slow every patient check-in/booking write, while asynchronous replication (scheduled DistCp or a streaming replication tool) keeps primary-site writes fast and accepts a small, bounded lag. Bandwidth: needs to sustain the ~10TB/day change volume plus peak-hour headroom and post-outage catch-up — a dedicated multi-Gbps link is recommended over best-effort internet transit. Cost: roughly doubles the storage footprint (a full second copy) plus the dedicated link and its own ops overhead — justified given queue/appointment continuity is patient-care-critical. RTO/RPO: asynchronous replication realistically gives an RTO of tens of minutes to a few hours (detect outage + redirect traffic) and an RPO equal to the replication lag (minutes), not the near-zero RPO a synchronous setup would give — the accepted trade-off for keeping local writes fast.

Scenario B: Federation for Scale-Out

The cluster needs to support 200M files (not possible with single NameNode due to heap limits). Design a Federation setup: how many NameNodes? How to distribute files across namespaces? What is the performance impact? Can you still run cluster-wide MapReduce jobs?

200M files needs $200M \div 1M \approx 200GB$ of NameNode heap on one NameNode — impractical, so Federation is required. Design: split the namespace across, e.g., 4 NameNodes each owning ~50M files (~50GB heap each), one per major Medi Queue AI region (North/South/East/West branch groups), all sharing the same DataNode pool via ViewFs/client-side mount tables. Performance: metadata operations scale roughly linearly with NameNode count since each namespace has its own heap and RPC queue, removing the single-NameNode bottleneck; per-request latency for a single path is unchanged since it still only touches the NameNode that owns it. Cluster-wide MapReduce: yes, but jobs spanning namespaces (e.g. a nationwide no-show report across North+South) must be ViewFs-aware and address paths like /North/... and /South/... explicitly — Federation scales metadata independently, not a single unified namespace.

Scenario C: Real-Time Alerting & Auto-Remediation

Design a monitoring + auto-remediation system: when DataNode disk usage exceeds 85%, automatically trigger: (1) alert to ops, (2) pause new block writes to that node, (3) trigger re-replication of critical blocks. What tools would you use? How to test safely?

Design: JMX exporters on each DataNode → Prometheus → Alertmanager. When disk usage crosses 85%, Alertmanager fires (1) an ops page/Slack alert, and (2) a webhook to a small remediation service that calls the HDFS admin/balancer API to stop the NameNode choosing that node for new block writes, and (3) triggers 'hdfs balancer' or targeted re-replication of that node's most under-replicated/critical blocks onto healthier nodes. Tools: Prometheus + JMX/node exporters, Alertmanager, a lightweight remediation runner (e.g. a script or StackStorm/Rundeck) calling HDFS admin APIs, visualized in Grafana. Testing safely: deploy in shadow/dry-run mode first (alerts fire, remediation actions are logged but not executed) against a staging cluster with synthetic disk-usage growth; only promote to full automatic execution after several dry-run cycles confirm it behaves correctly and doesn't itself cause instability (e.g. a thundering-herd re-replication overloading the network).

Scenario D: Cost Benchmarking & Optimization

The company wants to understand their true cost per TB stored. Include: hardware amortization, power, cooling, network, ops salary (allocate 10% of an engineer's time). Compare: (1) on-premise cluster, (2) AWS S3, (3) hybrid (hot on-premise + cold cloud). Which is most cost-effective for 100TB with 2-year horizon?

On-premise (100TB, 2 years): hardware amortized (~10 DataNode-class nodes ≈ \$5,000, spread over 2 years ≈ \$2,500/year) + power/cooling ($\approx 10 \times \$1,000/\text{year} = \$10,000/\text{year}$) + network + ops (allocating 10% of an engineer's time is often the largest line item once hardware is owned) — roughly \$200–\$400/TB/year once ops time is counted honestly. AWS S3: no hardware/power/cooling to manage; usage-based cost (roughly \$20–\$25/TB/month for standard storage, cheaper for infrequent-access/Glacier tiers), scales elastically with no capacity-planning risk, at the cost of ongoing fees and egress charges for frequent retrieval. Hybrid (hot on-prem + cold cloud): keeps frequently-queried recent queue data on fast on-prem storage while archiving older/audit-only data to S3/Glacier — captures most of the on-prem latency benefit for active queue operations while avoiding hot-tier prices for data nobody queries. Recommendation: hybrid — for the same reason tiering made sense earlier in this workbook, it matches storage cost to actual access pattern (recent data accessed constantly, data >30 days old rarely touched) rather than over-paying either extreme.

Criteria	Wt.	L4 (40–50)	L3 (30–39)	L2 (20–29)	L1 (0–19)
Monitoring & Observability	20%	Comprehensive metrics, realistic alert thresholds, actionable	Good metrics; some thresholds arbitrary	Basic metrics; incomplete alert strategy	Incomplete monitorin design
Cost Optimization	25%	Accurate cost breakdown, realistic savings targets, trade-offs justified	Good analysis; minor gaps in cost modeling	Basic cost analysis; limited optimization ideas	Incorrect or missing analysis
Backup & DR Planning	20%	Credible RTO/RPO targets, feasible failover design, HA understanding	Good DR plan; some RTO/RPO unrealistic	Basic backup strategy; weak failover plan	Incomplete or missin strategy
Scaling & Capacity Planning	20%	Data-driven forecasts, proactive scaling, Federation/HA impact analysis	Good capacity plan; some growth projections weak	Basic capacity analysis; limited forward planning	Missing or unrealistic capacity planning
Documentation & Clarity	15%	Clear dashboards, formulas, step-by-step reasoning, justified decisions	Mostly clear; minor formatting/detail gaps	Understandable but lacks design diagrams/detail	Unclear or poorly organized

Evaluation Rubric

Total Marks: 100 (20 + 25 + 20 + 20 + 15)